

Quarkus Flow

Orquestrando Complexidade e
Agentes de IA

Matheus Cruz

Todo o sistema começa
simples

Um endpoint, um banco de
dados e uma regra de
negócio

**Mas o software nunca
permanece simples...**

"Complexidade inerente ao Software"

Existem muitas variáveis que se combinam, muitos estados diferentes a serem lidados. Conforme o software cresce, atender a diferentes escalas de usuários introduz novos desafios.

– Brooks, Frederick. No Silver Bullet

O inevitável



O inevitável



O inevitável

- Retries



O inevitável

- Retries
- Eventos



O inevitável

- Retries
- Eventos
- Filas



O inevitável

- Retries
- Eventos
- Filas
- Timeout



O inevitável

- Retries
- Eventos
- Filas
- Timeout
- Compensação



O inevitável

- Retries
- Eventos
- Filas
- Timeout
- Compensação
- HITL (Human-in-the-loop)



O inevitável

- Retries
- Eventos
- Filas
- Timeout
- Compensação
- HITL (Human-in-the-loop)
- Integração com sistemas legados



Cuidado com o Workflow engine escondido

Cuidado com o Workflow engine escondido

- Se você não tomar cuidado a lógica de coordenação vai, aos poucos, se espalhando por diversas partes do seu código

Cuidado com o Workflow engine escondido

- Se você não tomar cuidado a lógica de coordenação vai, aos poucos, se espalhando por diversas partes do seu código
- Com o tempo, a lógica de negócio e as preocupações de infraestrutura acabam ficando profundamente acopladas

Cuidado com o Workflow engine escondido

- Se você não tomar cuidado a lógica de coordenação vai, aos poucos, se espalhando por diversas partes do seu código
- Com o tempo, a lógica de negócio e as preocupações de infraestrutura acabam ficando profundamente acopladas
- O código fica mais difícil de receber manutenção e surge aquela frase: "Precisamos refatorar o projeto"



A scene from the TV show Survivor. In the foreground, several contestants are covered in mud and are pushing a large, heavy, woven basket filled with mud. In the background, other contestants stand near a body of water, with a dense forest behind them. Two red banners are visible, one with the word 'SURVIVOR' and another with 'OUTWIT'.

Grande bola de lama (Big Ball of Mud)

Exemplo: Compensação na Grande Bola de Lama

```
1 public class OrderService {
2     @Inject RestClient stockClient;
3     @Inject RestClient paymentClient;
4     @Inject RestClient shippingClient;
5     @Inject EntityManager em;
6     @Inject Mailer mailer;
7
8     public void processOrder(Order order) {
9         try {
10             // Mutate order state - PENDING
11             order.setStatus("PENDING");
12             em.persist(order);
13
14             // HTTP call to stock service
15             var stockResponse = stockClient.post("/api/stoc
```

Exemplo: Compensação na Grande Bola de Lama

```
1 public class OrderService {
2     @Inject RestClient stockClient;
3     @Inject RestClient paymentClient;
4     @Inject RestClient shippingClient;
5     @Inject EntityManager em;
6     @Inject Mailer mailer;
7
8     public void processOrder(Order order) {
9         try {
10             // Mutate order state - PENDING
11             order.setStatus("PENDING");
12             em.persist(order);
13
14             // HTTP call to stock service
15             var stockResponse = stockClient.post("/api/stoc
```

Exemplo: Compensação na Grande Bola de Lama

```
6      @Inject Mailer mailer;
7
8      public void processOrder(Order order) {
9          try {
10             // Mutate order state - PENDING
11             order.setStatus("PENDING");
12             em.persist(order);
13
14             // HTTP call to stock service
15             var stockResponse = stockClient.post("/api/stoc
16                 .body(order.getItems()).invoke();
17             order.setStockReservationId(stockResponse.getId
18             order.setStatus("STOCK_RESERVED");
19             em.merge(order);
20
```

Exemplo: Compensação na Grande Bola de Lama

```
20
21      // HTTP call to payment service
22      var paymentResponse = paymentClient.post("/api/
23          .header("X-Customer-Id", order.getCustomerId())
24          .body(Map.of("amount", order.getTotalAmount()))
25          .invoke();
26
27      if (paymentResponse.getStatus() != 200) {
28          // Compensate: release stock via HTTP
29          stockClient.post("/api/stock/release")
30              .body(order.getStockReservationId()).invoke();
31          order.setStatus("PAYMENT_FAILED");
32          em.merge(order);
33          mailer.send(order.getCustomerEmail(), "Payment failed");
34          throw new PaymentException("Payment failed")
```


Exemplo: Compensação na Grande Bola de Lama

```
37         order.setPaymentId(paymentResponse.getPaymentId());
38         order.setStatus("PAYMENT_CONFIRMED");
39         em.merge(order);
40
41         // HTTP call to shipping service
42         var shippingResponse = shippingClient.post("/api/shipping")
43             .body(Map.of("address", order.getShippingAddress(),
44                           "items", order.getItems()))
45             .invoke();
46
47         if (shippingResponse.getStatus() != 200) {
48             // Compensate: refund payment via HTTP
49             paymentClient.post("/api/payment/refund")
50                 .body(order.getPaymentId()).invoke();
51             // Compensate: release stock via HTTP
```

Exemplo: Compensação na Grande Bola de Lama

```
54         order.setStatus("SHIPPING_FAILED");
55         em.merge(order);
56         mailer.send(order.getCustomerEmail(), "Ship
57         throw new ShippingException("Shipping faile
58     }
59
60     order.setShippingId(shippingResponse.getTrackin
61     order.setStatus("CONFIRMED");
62     em.merge(order);
63     mailer.send(order.getCustomerEmail(), "Order co
64
65     } catch (Exception e) {
66         // What if HTTP calls fail?
67         // What if compensation HTTP calls fail?
68         // What about network timeouts?
```

Exemplo: Compensação na Grande Bola de Lama

```
63         matter.send(order.getCustomerEmail(), order.cc);
64
65     } catch (Exception e) {
66         // What if HTTP calls fail?
67         // What if compensation HTTP calls fail?
68         // What about network timeouts?
69         // What about partial failures?
70         // How to monitor this mess?
71         order.setStatus("ERROR");
72         em.merge(order);
73         log.error("Order processing failed", e);
74         throw new OrderProcessingException(e);
75     }
76 }
77 }
```

**Você acaba de forma
implícita, criando um
workflow engine
manualmente**

**E se a orquestração fosse
explícita?**



Quarkus Flow

O que é um Workflow?

"A automação de um processo, onde os dados são passados de uma tarefa para outra para execução, seguindo um conjunto de regras."

O que ele veio resolver?

Workflow surgiu para resolver a coordenação de processos complexos com múltiplas etapas, regras, estados e integrações distribuídas. Ele evita que fluxos de negócio fiquem espalhados em `if/else`, filas, cron jobs e tabelas de controle difíceis de manter, monitorar e recuperar após falhas.

Quarkus Flow

Quarkus Flow é Workflow Engine para Quarkus:

Quarkus Flow

Quarkus Flow é Workflow Engine para Quarkus:

- Gerenciamento de estado, controle de fluxo, chamadas remotas, mensageria, resiliência, observabilidade, etc.

Quarkus Flow

Quarkus Flow é Workflow Engine para Quarkus:

- Gerenciamento de estado, controle de fluxo, chamadas remotas, mensageria, resiliência, observabilidade, etc.
- Permite Human-in-the-Loop (HITL) e Agentic patterns

Quarkus Flow

Quarkus Flow é Workflow Engine para Quarkus:

- Gerenciamento de estado, controle de fluxo, chamadas remotas, mensageria, resiliência, observabilidade, etc.
- Permite Human-in-the-Loop (HITL) e Agentic patterns
- Permite a criação de workflows duráveis (Durable Workflows)

Quarkus Flow

Quarkus Flow é Workflow Engine para Quarkus:

- Gerenciamento de estado, controle de fluxo, chamadas remotas, mensageria, resiliência, observabilidade, etc.
- Permite Human-in-the-Loop (HITL) e Agentic patterns
- Permite a criação de workflows duráveis (Durable Workflows)
- Deixa sua lógica de negócio limpa e orquestra a complexidade para você

Serverless Workflow Specification

Serverless Workflow Specification

- Projeto CNCF Sandbox (candidatando-se à incubação este ano)

Serverless Workflow Specification

- Projeto CNCF Sandbox (candidatando-se à incubação este ano)
- Ecossistema neutro em relação a fornecedores (vendor-neutral)

Serverless Workflow Specification

- Projeto CNCF Sandbox (candidatando-se à incubação este ano)
- Ecossistema neutro em relação a fornecedores (vendor-neutral)
- Integração fluida com arquiteturas modernas

Serverless Workflow Specification

- Projeto CNCF Sandbox (candidatando-se à incubação este ano)
- Ecossistema neutro em relação a fornecedores (vendor-neutral)
- Integração fluida com arquiteturas modernas
- Padrões suportados: CloudEvents, AsyncAPI, gRPC, OpenAPI, A2A, MCP e outros

Java DSL

```
1 @ApplicationScoped
2 public class EmitFlow extends Flow {
3     @Override
4     public Workflow descriptor() {
5         return workflow("emit", "test")
6             .tasks(
7                 emit("com.petstore.order.placed.v1")
8                     .source("https://petstore.com")
9                     .jsonData(Order.class))
10            .build();
11     }
12 }
```

Java DSL

```
1 @ApplicationScoped
2 public class EmitFlow extends Flow {
3     @Override
4     public Workflow descriptor() {
5         return workflow("emit", "test")
6             .tasks(
7                 emit("com.petstore.order.placed.v1")
8                     .source("https://petstore.com")
9                     .jsonData(Order.class))
10            .build();
11     }
12 }
```

Java DSL

```
1  @ApplicationScoped
2  public class EmitFlow extends Flow {
3      @Override
4      public Workflow descriptor() {
5          return workflow("emit", "test")
6              .tasks(
7                  emit("com.petstore.order.placed.v1")
8                      .source("https://petstore.com")
9                      .jsonData(Order.class))
10         .build();
11     }
12 }
```

YAML Definition

```
1 document:
2   dsl: '1.0.3'
3   namespace: test
4   name: emit
5   version: '0.1.0'
6   do:
7     - emitEvent:
8       emit:
9         event:
10          with:
11            source: https://petstore.com
12            type: com.petstore.order.placed.v1
13            data:
14              client:
15                firstName: Cruella
```

YAML Definition

```
1 document:
2   dsl: '1.0.3'
3   namespace: test
4   name: emit
5   version: '0.1.0'
6   do:
7     - emitEvent:
8       emit:
9         event:
10          with:
11            source: https://petstore.com
12            type: com.petstore.order.placed.v1
13            data:
14              client:
15                firstName: Cruella
```

YAML Definition

```
1 document:
2   dsl: '1.0.3'
3   namespace: test
4   name: emit
5   version: '0.1.0'
6   do:
7     - emitEvent:
8       emit:
9         event:
10          with:
11            source: https://petstore.com
12            type: com.petstore.order.placed.v1
13            data:
14              client:
15                firstName: Cruella
```


YAML Definition

```
1 document:
2   dsl: '1.0.3'
3   namespace: test
4   name: emit
5   version: '0.1.0'
6   do:
7     - emitEvent:
8       emit:
9         event:
10          with:
11            source: https://petstore.com
12            type: com.petstore.order.placed.v1
13            data:
14              client:
15                firstName: Cruella
```

YAML Definition

```
5 version: 0.1.0
6 do:
7   - emitEvent:
8     emit:
9       event:
10        with:
11          source: https://petstore.com
12          type: com.petstore.order.placed.v1
13          data:
14            client:
15              firstName: Cruella
16              lastName: de Vil
17            items:
18              - breed: dalmatian
19                quantity: 101
```

Arquitetura e Ecossistema

Integração nativa com a infraestrutura Quarkus

Arquitetura plugável

Quarkus Flow

Java SDK / DSL (Engine)

Persistence

Redis/JPA

Messaging

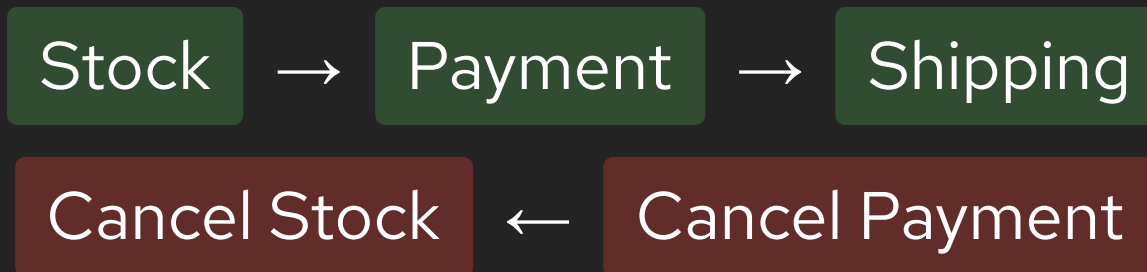
Any

**Smallrye Fault
Tolerance**

Quarkus Infrastructure (Arc, HTTP, etc.)

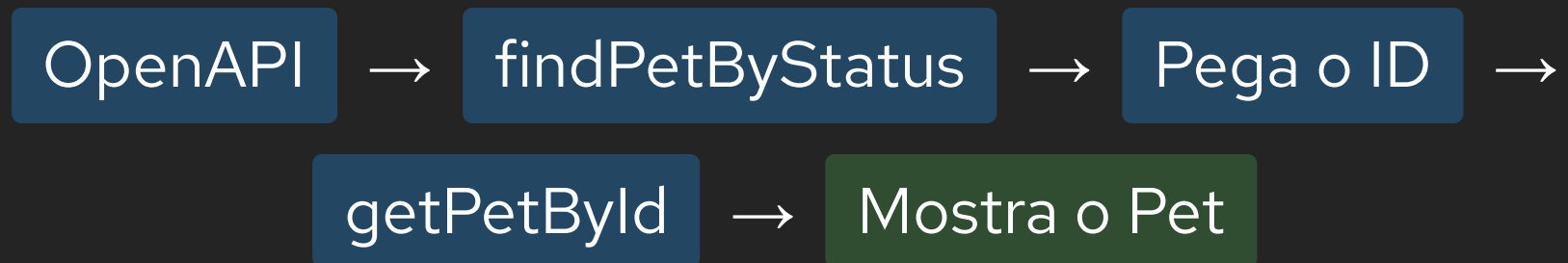
Demo: Compensation pattern

- Try/Catch utilizando o Java DSL
- Utilizar catchWhen com ProblemDetails (RFC 7807)
- then e utilização de diretivas



Demo: Petstore OpenAPI

- Consumir um documento OpenAPI
- Executar uma de suas operações
- Gerenciamento de estado
- Encadeamento de chamadas HTTP através do mesmo documento OpenAPI



Integração com Langchain4J

Workflow

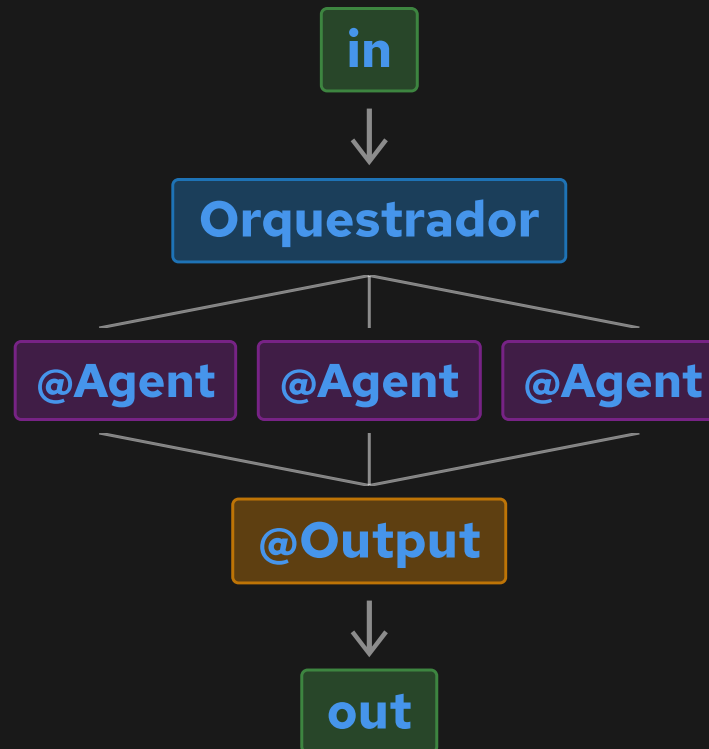
Agents são programaticamente orquestrados através de um caminho predefinido

Agents

Agents direcionam dinamicamente sua própria ordem de invocação e têm controle sobre como executam as tarefas

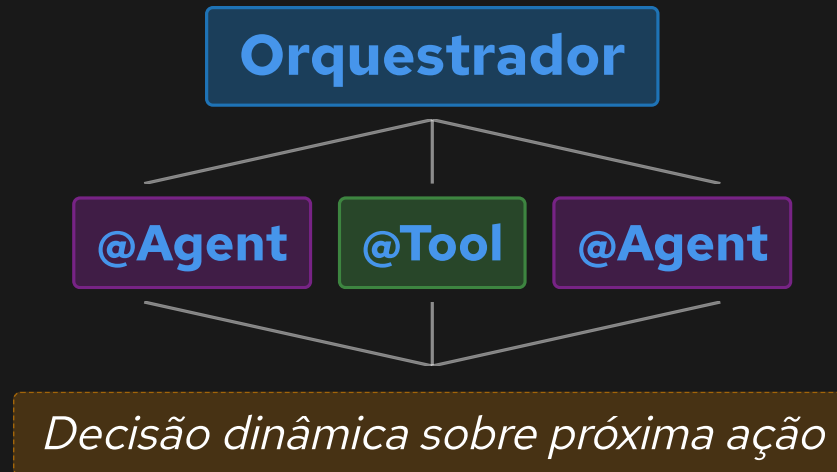
Workflow

Agents são programaticamente orquestrados através de um caminho predefinido



Agents

Agents direcionam dinamicamente sua própria ordem de invocação



Tipos de Agents Workflows

Tipos de Agents Workflows

- `@SequenceAgent` permite você executar agentes de forma sequencial

Tipos de Agents Workflows

- `@SequenceAgent` permite você executar agentes de forma sequencial
- `@ParallelAgent` permite você executar diversos agentes ao mesmo tempo

Tipos de Agents Workflows

- `@SequenceAgent` permite você executar agentes de forma sequencial
- `@ParallelAgent` permite você executar diversos agentes ao mesmo tempo
- `@LoopAgent` como um for-loop no Java, permite você executar agentes até uma dada condição for satisfeita

Tipos de Agents Workflows

- `@SequenceAgent` permite você executar agentes de forma sequencial
- `@ParallelAgent` permite você executar diversos agentes ao mesmo tempo
- `@LoopAgent` como um for-loop no Java, permite você executar agentes até uma dada condição for satisfeita
- `@ConditionalAgent` permite você dado uma condição utilizar ou não um agente

Tipos de Agents Workflows

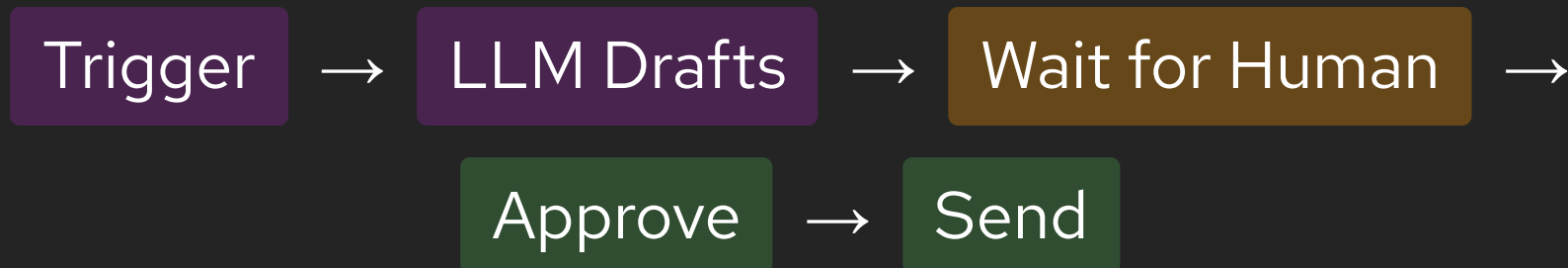
- `@SequenceAgent` permite você executar agentes de forma sequencial
- `@ParallelAgent` permite você executar diversos agentes ao mesmo tempo
- `@LoopAgent` como um for-loop no Java, permite você executar agentes até uma dada condição for satisfeita
- `@ConditionalAgent` permite você dado uma condição utilizar ou não um agente
- `@HumanInTheLoop` permite de forma programática pegar um input ou feedback de um humano

Tipos de Agents Workflows

- `@SequenceAgent` permite você executar agentes de forma sequencial
- `@ParallelAgent` permite você executar diversos agentes ao mesmo tempo
- `@LoopAgent` como um for-loop no Java, permite você executar agentes até uma dada condição for satisfeita
- `@ConditionalAgent` permite você dado uma condição utilizar ou não um agente
- `@HumanInTheLoop` permite de forma programática pegar um input ou feedback de um humano
- `Non-AI agents` permite com que você escreva código para performar alguma tarefa

Demo: Agentic Workflows com LangChain4j

- Gerador de newsletter (LangChain4j)
- Workflow engine orquestrando chamadas para LLM
- Human-in-the-loop (HITL)
- Durable workflows
- Observabilidade



Experimente!

- [Quarkus Flow Documentation](#)
- [Quarkus Flow GitHub](#)
- [Quarkus Flow Workshop](#)
- [Demos](#)
- [Serverless Workflow Specification](#)

Matheus Cruz

- Software Engineer na IBM
- Quarkus Flow contributor
- CNCF Serverless Workflow Specification (SDK Java) contributor
- Quarkus Club Coordinator



Sorteio

Deixe um star para participar do sorteio